Predictive Maintenance Digital Twin Pipeline

Phase 1 walkthrough: raw sensor data → features → simulated federated sites → one trained digital twin per site.

This document explains, in plain language, what each stage of the pipeline does, why it's built that way, and what its output actually looked like when run on your real data. It follows the five scripts in **scripts/**, in order.

The big picture

A "digital twin" here is just a small neural network that watches a machine's recent sensor readings and predicts its Remaining Useful Life (RUL) — how much longer it has before it fails. Phase 1 builds one such model per simulated "federated site" (a data source that would, in a real deployment, be a separate factory or fleet that doesn't share raw data with anyone else). We use two public datasets to simulate seven such sites:

• **PRONOSTIA/FEMTO** — vibration sensor recordings from bearings run to failure, split into 3 sites (condition_1/2/3) by test rig operating condition.
• **NASA C-MAPSS** — simulated turbofan engine sensor readings, already split into 4 sites (FD001–FD004) by operating/fault condition.

Step 1 — Turning raw sensor files into features

Script: scripts/01_preprocess.py
Why: Raw vibration data is 2,560 numbers per file, 25,000+ files. A neural network could theoretically learn from that directly, but it's overkill for Phase 1 and expensive to compute. Instead we compress each file down to 10 numbers that summarize "how rough is this vibration signal right now": for both the horizontal and vertical accelerometer axes we compute the RMS (average energy), Peak (worst spike), Kurtosis (how spiky/impulsive the signal is — a classic early-warning sign of bearing damage), Skew, and Standard Deviation. Each row also gets a RUL label: "how many more files until this bearing fails," counting down to exactly 0 at the last file we have for that bearing.

For C-MAPSS the raw data is already tabular (21 sensor readings per engine cycle), so the job is simpler: label every row with RUL (capped at 125 cycles, a standard convention meaning "more than 125 cycles left" all counts the same, since the very early life of an engine looks the same either way), and rescale every sensor column into a common 0-1 range so no single sensor dominates just because it's measured in bigger numbers.

One real snag we hit here: a handful of the FEMTO files (e.g. Bearing1_4) use a semicolon instead of a comma between numbers — a known quirk of this public dataset. The script sniffs each file's delimiter before reading it so this doesn't silently corrupt those rows.

Output:

Output file	Rows	Detail
femto_features_labeled.csv	24,889	bearings: condition_1=7, condition_2=7, condition_3=3
cmapss_train_labeled.csv	160,359	engine units: FD001=100, FD002=260, FD003=100, FD004=249
cmapss_test_labeled.csv	104,897	engine units: FD001=100, FD002=259, FD003=100, FD004=248

Sanity check that matters most here: for every bearing/engine, RUL actually counts down to 0 at its last recorded row (verified for one example per site when the script runs) — if it didn't, every later step would be training on mislabeled data.

Step 2 — Splitting into per-site train/test folders

Script: scripts/02_federated_split.py
Why: In federated learning, each site trains only on its own data. So the labeled rows from Step 1 get divided into 7 folders (federated_clients/<site>/), one per site, each with its own train.csv and test.csv.

The one rule that actually matters here: the split is done by **whole bearing/engine**, never by row. If you split by row instead, you'd end up with, say, cycles 1-80 of engine #3 in the training set and cycles 81-120 of that *same* engine in the test set — the model would have already "seen" that engine's degradation pattern, making the test score meaninglessly optimistic. Splitting by whole machine (roughly 75% of machines to train, 25% held out for test) avoids that leak.

Site	Bearings/units	Train rows	Test rows
FD001	100	15,471	5,160
FD002	260	40,461	13,298
FD003	100	18,168	6,552
FD004	249	45,959	15,290
condition_1	7	9,396	5,251
condition_2	7	6,204	1,452
condition_3	3	2,071	515

Step 3 — The digital twin model

Script: scripts/03_model.py
Why this shape: The model looks at a sliding window of the last 10 timesteps of features (not just the single most recent one) so it can see a *trend*, not just a snapshot — a bearing that's rapidly getting rougher looks different from one that's steady even if their most recent reading is identical. It's built from three pieces stacked together:
• **Conv1D layers** (32 then 64 filters) act like a pattern detector, scanning the window for local shapes in the sensor trend.
• An **LSTM layer** (64 units) reads the window in order and summarizes the overall degradation trend into one vector.

• Two small **Dense layers** turn that vector into a single RUL number. The layer just before the final number is also returned separately as an "embedding" — not used yet, but reserved for later phases of this research project.

Why the size matters: in a real federated setup this model gets sent back and forth over the network every training round, so a bigger model directly means more network traffic. It's kept under ~500,000 parameters for that reason.

Output: 42,613 parameters (well under the 500,000 budget), window size = 10 timesteps.

Step 4 — Training one model per site

Script: scripts/04_train.py
Why: Each site trains its own model independently — no mixing data across sites (that's intentionally out of scope for Phase 1; combining what different sites learn is the job of federated *aggregation*, a later phase). For each site, the script builds sliding windows from that site's own train.csv (never crossing from one bearing/engine into another), trains with the Adam optimizer to minimize prediction error, and stops automatically once the test score stops improving for 10 epochs in a row ("early stopping") — this avoids overfitting and avoids wasting time training past the point of improvement.

A real bug we found and fixed along the way

The first training run looked broken for all 4 C-MAPSS sites: the error score barely moved from the very first epoch onward, as if the model had simply given up and started guessing "the average RUL" for every single engine regardless of its actual sensor readings. Digging in: Step 1's sensor scaling (required, and correct) squeezes every sensor into the 0-1 range, but several of the most informative sensors happen to live in a very narrow slice of that range in practice (e.g. 0.95 to 0.99, not 0 to 1). That means the real, useful signal is a tiny wobble sitting on top of a value close to 1 — and the network's first layer wasn't rescaling that wobble to something it could actually respond to, so it defaulted to ignoring the input almost entirely.

The fix was to add one extra layer (a "batch normalization" layer) right at the model's input, which automatically re-centers and rescales each sensor so the real signal is no longer drowned out. After that one change, the C-MAPSS sites went from "not learning at all" to error scores in line with what's normally reported in the research literature for this dataset.

Output (RMSE = typical prediction error, in the same units as RUL — lower is better):

Site	Training windows	Final test RMSE
FD001	14,796	15.5
FD002	38,706	19.3
FD003	17,493	16.0
FD004	44,276	19.0
condition_1	9,351	614.9
condition_2	6,159	214.8
condition_3	2,053	139.9

Step 5 — Did each model actually learn something?

Script: scripts/05_verify.py
Why: A low-looking error score can be misleading on its own — the real question is whether the model beats the laziest possible guess: always predicting that site's average RUL. This step computes that "constant-mean baseline" for each site and flags any site whose model doesn't clearly beat it (by at least 20%) as "failed to converge."

Site	Model RMSE	Baseline RMSE	Verdict
FD001	15.5	41.7	OK
FD002	19.3	41.7	OK
FD003	16.0	40.1	OK
FD004	19.0	40.6	OK
condition_1	614.9	768.2	Did not converge
condition_2	214.8	210.3	Did not converge
condition_3	139.9	148.7	Did not converge

Reading this table: all 4 C-MAPSS sites clearly beat their baseline — the models learned a real, useful relationship between sensor trends and remaining life. The 3 FEMTO sites (condition_1/2/3) are flagged, and that's a real finding worth understanding rather than a bug to silently patch over: each FEMTO site only has 3-7 bearings total (2-3 held out for testing), and their RUL label is a raw file count that ranges from 0 to nearly 2,800 depending on which specific bearing you're looking at, with no upper cap. A 10-step window of vibration statistics has no way of knowing *which* bearing it's looking at or that bearing's total lifespan, so predicting the exact remaining count is a much harder problem here than it is for C-MAPSS (where RUL is capped at 125 and there are far more machines to learn from). If better FEMTO performance matters for later phases of the project, the standard fix in the literature is to predict RUL as a *fraction of that bearing's total life* instead of a raw file count — a labeling change, not a bug fix, so it was left as a decision for later rather than made silently here.

Chart: the same numbers, plus the training curves that produced them (how the error score changed epoch by epoch).

